Lecture 3. Data Transformation

dplyr: filter, arrange, select, mutate, summarise, group_by, the pipe

Contents

3.1 Why transform data **2**
3.1.1 The five verbs . . . 2

3.2 Filter rows with filter() **2**
3.2.1 Comparisons . . . 3
3.2.2 Logical operators . . . 3
3.2.3 Missing values . . . 3

3.3 Arrange rows with arrange() **4**

3.4 Select columns with select() **4**

3.5 Add new variables with mutate() **5**
3.5.1 Useful creation functions . . . 5

3.6 Grouped summaries with summarise() **6**
3.6.1 The pipe . . . 6
3.6.2 Missing values and counts . . . 7
3.6.3 Useful summary functions . . . 8

3.7 Grouped mutates and filters **8**

3.1 Why transform data

Motivation. dplyr turns each data manipulation into one verb; a chain of verbs answers most descriptive questions.

Data: `flights` from `nycflights13`, all 336 776 flights that left New York City in 2013.

```
library(nycflights13)
library(tidyverse)
```

```
flights
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Column types: `int`, `dbl` (real), `chr`, `dttm` (date-time); later `lgl`, `fctr`, `date`.

3.1.1 The five verbs

verb	action
`filter()`	pick observations (rows) by their values
`arrange()`	reorder rows
`select()`	pick variables (columns) by their names
`mutate()`	create new variables from existing ones
`summarise()`	collapse many values to a single summary

First argument a data frame, then what to do with unquoted variable names; the result is a new data frame. `group_by()` makes every verb act group by group.

3.2 Filter rows with filter()

Motivation. Restrict to the relevant cases, with care for real numbers and missing values.

```
jan1 <- filter(flights, month == 1, day == 1)
jan1
```

```
## # A tibble: 842 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     1      517            515         2      830            819        11
## 2  2013     1     1      533            529         4      850            830        20
## 3  2013     1     1      542            540         2      923            850        33
## 4  2013     1     1      544            545        -1     1004           1022       -18
## 5  2013     1     1      554            600        -6      812            837       -25
## # i 837 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

Commas mean "and". dplyr never modifies its input: save with `<-`; `(jan1 <- ...)` also prints.

3.2.1 Comparisons

`>`, `>=`, `<`, `<=`, `!=`, `==`. `filter(flights, month = 1)` is an error (suggests `month == 1`).

Doubles have finite precision:

```
sqrt(2) ^ 2 == 2
```
```
## [1] FALSE
```

```
1 / 49 * 49 == 1
```
```
## [1] FALSE
```

```
near(sqrt(2) ^ 2, 2)
```
```
## [1] TRUE
```

Compare reals with `near()` (a tolerance), not `==`.

Why $1/49 \cdot 49 \neq 1$ [Supplement]

With a 53-bit mantissa $1/49$ is rounded; multiplying by 49 keeps the error, off by one last bit:

```
print(1 / 49 * 49, digits = 17)
```
```
## [1] 0.99999999999999989
```

`near(x, y)` returns `abs(x - y) < tol` with `tol` about 1.5×10^{-8} (the square root of machine epsilon).

3.2.2 Logical operators

`&` is "and", `|` is "or", `!` is "not"; `xor()` is exclusive or. Flights in November or December:

```
filter(flights, month == 11 | month == 12)
filter(flights, month %in% c(11, 12))      # the same, shorter
```

`filter(flights, month == 11 | 12)` means `(month == 11) | 12`, and `12` is `TRUE`: every flight is kept. By De Morgan, $!(x \,\&\, y) = !x \,|\, !y$, $!(x \,|\, y) = !x \,\&\, !y$, "delayed at most two hours on arrival and departure" is

```
filter(flights, !(arr_delay > 120 | dep_delay > 120))
filter(flights, arr_delay <= 120, dep_delay <= 120)
```

Make complicated conditions explicit variables first. `&&`, `||` take single values (for `if`, Lecture 10).

3.2.3 Missing values

`NA` is an unknown value, and it is contagious:

```
NA > 5
```
```
## [1] NA
```

```
NA + 10
```
```
## [1] NA
```

```
NA == NA
```
```
## [1] NA
```

Two unknown ages may or may not be equal, so `NA == NA` is `NA`; test with `is.na()`.

`filter()` keeps `TRUE` rows, dropping `FALSE` and `NA`; keep `NA` explicitly:

```
df <- tibble(x = c(1, NA, 3))
filter(df, x > 1)
```

```
## # A tibble: 1 x 1
##       x
##   <dbl>
## 1     3
```

```
filter(df, is.na(x) | x > 1)
```

```
## # A tibble: 2 x 1
##       x
##   <dbl>
## 1    NA
## 2     3
```

3.3 Arrange rows with arrange()

`arrange()` sorts by columns, later ones breaking ties; `desc()` reverses. `NA` always goes last.

```
arrange(flights, desc(dep_delay))
```

```
## # A tibble: 336,776 x 19
##    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay
##   <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl>
## 1  2013     1     9      641            900      1301     1242           1530      1272
## 2  2013     6    15     1432           1935      1137     1607           2120      1127
## 3  2013     1    10     1121           1635      1126     1239           1810      1109
## 4  2013     9    20     1139           1845      1014     1457           2210      1007
## 5  2013     7    22      845           1600      1005     1044           1815       989
## # i 336,771 more rows
## # i 10 more variables: carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>,
## #   time_hour <dttm>
```

```
df <- tibble(x = c(5, 2, NA))
arrange(df, desc(x))
```

```
## # A tibble: 3 x 1
##       x
##   <dbl>
## 1     5
## 2     2
## 3    NA
```

3.4 Select columns with select()

Motivation. Narrow hundreds of variables down to the few that matter.

```
select(flights, year, month, day)          # by name
select(flights, year:day)                  # all columns between year and day
select(flights, -(year:day))               # all except those
select(flights, seq(1, 10, by = 2))        # by position
select(flights, time_hour, air_time, everything())   # move to the front
rename(flights, tail_num = tailnum)        # rename, keep all others
```

Helper functions select by name pattern: `starts_with("abc")`, `ends_with("xyz")`, `contains("ijk")`, `matches("(.)\textbackslash \textbackslash 1")` (a regular expression, Lecture 7) and `num_range("x", 1:3)` for `x1`, `x2`, `x3`.

3.5 Add new variables with mutate()

Motivation. Compute unrecorded quantities (speed, time gained) from existing columns.

`mutate()` adds columns at the end; start narrow to see them:

```
flights_sml <- select(flights, year:day, ends_with("delay"), distance, air_time)
mutate(flights_sml,
  gain = dep_delay - arr_delay,
  hours = air_time / 60,
  gain_per_hour = gain / hours
)
```

```
## # A tibble: 336,776 x 10
##    year month   day dep_delay arr_delay distance air_time  gain hours gain_per_hour
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl> <dbl> <dbl>          <dbl>
## 1  2013     1     1         2        11     1400      227    -9  3.78          -2.38
## 2  2013     1     1         4        20     1416      227   -16  3.78          -4.23
## 3  2013     1     1         2        33     1089      160   -31  2.67         -11.6
## 4  2013     1     1        -1       -18     1576      183    17  3.05           5.57
## 5  2013     1     1        -6       -25      762      116    19  1.93           9.83
## # i 336,771 more rows
```

New columns are usable in the same call; `transmute()` keeps only new ones.

3.5.1 Useful creation functions

Must be **vectorised**: vector in, vector of the same length out.

- Arithmetic `+ - * / ^` with recycling: `air_time / 60`, `x / sum(x)`, `y - mean(y)`.
- `%/%` (integer division), `%%` (remainder); times are `HHMM` numbers:

```
transmute(flights, dep_time, hour = dep_time %/% 100, minute = dep_time %% 100)
```

```
## # A tibble: 336,776 x 3
##   dep_time  hour minute
##      <int> <dbl>  <dbl>
## 1      517     5     17
## 2      533     5     33
## 3      542     5     42
## 4      544     5     44
## 5      554     5     54
## # i 336,771 more rows
```

- `log()`, `log2()`, `log10()` for data over orders of magnitude; `log2`: one unit = doubling.
- Offsets `lag()`, `lead()`; cumulative `cumsum()`, `cumprod()`, `cummin()`, `cummax()`, `cummean()`:

```
x <- 1:10
lag(x)
```

```
##  [1] NA  1  2  3  4  5  6  7  8  9
```

```
cumsum(x)
```

```
##  [1]  1  3  6 10 15 21 28 36 45 55
```

```
cummean(x)
```

```
##  [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5
```

- `min_rank()`: ties get the smallest rank; also `row_number()`, `dense_rank()`, `percent_rank()`, `cume_dist()`, `ntile()`.

```
y <- c(1, 2, 2, NA, 3, 4)
min_rank(y)
```

```
## [1]  1  2  2 NA  4  5
```

```
min_rank(desc(y))
```

```
## [1]  5  3  3 NA  2  1
```

3.6 Grouped summaries with summarise()

Motivation. Summaries compared across groups: group_by() plus summarise().

`summarise()` collapses a data frame to one row:

```
summarise(flights, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 1 x 1
##   delay
##   <dbl>
## 1  12.6
```

`group_by()` changes the unit of analysis to groups; it changes only how verbs act, not the data:

```
by_day <- group_by(flights, year, month, day)
summarise(by_day, delay = mean(dep_delay, na.rm = TRUE))
```

```
## # A tibble: 365 x 4
## # Groups:   year, month [12]
##    year month   day delay
##   <int> <int> <int> <dbl>
## 1  2013     1     1 11.5
## 2  2013     1     2 13.9
## 3  2013     1     3 11.0
## 4  2013     1     4  8.95
## 5  2013     1     5  5.73
## # i 360 more rows
```

Each `summarise()` peels off one grouping level (the message).

3.6.1 The pipe

Distance versus average delay per destination, with intermediate objects:

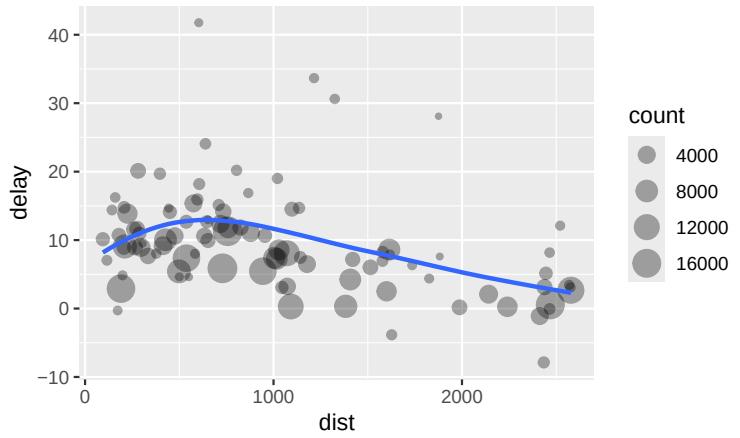
```
by_dest <- group_by(flights, dest)
delay <- summarise(by_dest, count = n(),
  dist = mean(distance, na.rm = TRUE), delay = mean(arr_delay, na.rm = TRUE))
delay <- filter(delay, count > 20, dest != "HNL")
```

The **pipe**: `x %>% f(y)` is `f(x, y)`; read "then":

```
delays <- flights %>%
  group_by(dest) %>%
  summarise(count = n(),
            dist = mean(distance, na.rm = TRUE),
            delay = mean(arr_delay, na.rm = TRUE)) %>%
  filter(count > 20, dest != "HNL")

ggplot(data = delays, mapping = aes(x = dist, y = delay)) +
```

```
  geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
```



Delay rises with distance up to about 750 miles, then falls (time made up in the air?). ggplot2 layers use `+`, not the pipe.

3.6.2 Missing values and counts

Without `na.rm = TRUE` one `NA` makes the summary `NA`. Missing delays are cancelled flights; remove them once:

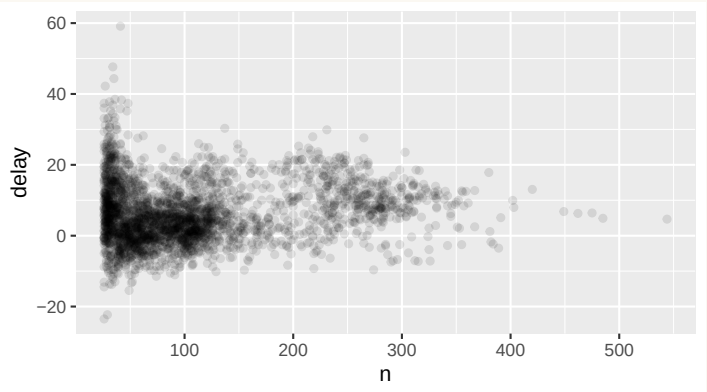
```
not_cancelled <- flights %>% filter(!is.na(dep_delay), !is.na(arr_delay))
```

Always include a count, `n()` or `sum(!is.na(x))`: averages of few observations vary much more.

Small groups have extreme averages [Supplement]

Average delay per plane against its number of flights is a funnel: $\mathrm{Var}(\bar{X}) = \sigma^2/n$, not a property of the planes. Dropping small groups removes the noise:

```
not_cancelled %>%
  group_by(tailnum) %>%
  summarise(delay = mean(arr_delay), n = n()) %>%
  filter(n > 25) %>%
  ggplot(mapping = aes(x = n, y = delay)) +
  geom_point(alpha = 1/10)
```



3.6.3 Useful summary functions

- Location: `mean(x)`, `median(x)`; `mean(arr_delay[arr_delay > 0])` is the average positive delay.
- Spread: `sd(x)`; robust `IQR(x)`, `mad(x)`.
- Rank: `min(x)`, `quantile(x, 0.25)`, `max(x)`.
- Position: `first(x)`, `nth(x, 2)`, `last(x)`.
- Counts: `n()`, `sum(!is.na(x))`, `n_distinct(x)`; `count(dest)` groups and counts; `count(tailnum, wt = distance)` sums a weight (miles per plane).
- Logicals: `TRUE`= 1, `FALSE`= 0, so `sum(x > 10)` counts and `mean(y == 0)` is a proportion.

```
not_cancelled %>%
  group_by(dest) %>%
  summarise(carriers = n_distinct(carrier)) %>%
  arrange(desc(carriers))
```

```
## # A tibble: 104 x 2
##   dest  carriers
##   <chr>    <int>
## 1 ATL          7
## 2 BOS          7
## 3 CLT          7
## 4 ORD          7
## 5 TPA          7
## # i 99 more rows
```

```
not_cancelled %>%
  group_by(year, month, day) %>%
  summarise(n_early = sum(dep_time < 500),     # flights before 5am
            hour_prop = mean(arr_delay > 60))  # share delayed over an hour
```

```
## # A tibble: 365 x 5
## # Groups:   year, month [12]
##    year month   day n_early hour_prop
##   <int> <int> <int>   <int>     <dbl>
## 1  2013     1     1       0    0.0722
## 2  2013     1     2       3    0.0851
## 3  2013     1     3       4    0.0567
## 4  2013     1     4       3    0.0396
## 5  2013     1     5       3    0.0349
## # i 360 more rows
```

`ungroup()` removes grouping.

3.7 Grouped mutates and filters

Under grouping, `rank()`, `n()`, `mean()` in `filter()` and `mutate()` act within groups.

```
flights_sml %>%                          # the worst members of each group
  group_by(year, month, day) %>%
  filter(rank(desc(arr_delay)) < 10)
```

```
## # A tibble: 3,306 x 7
## # Groups:   year, month, day [365]
##    year month   day dep_delay arr_delay distance air_time
##   <int> <int> <int>     <dbl>     <dbl>    <dbl>    <dbl>
## 1  2013     1     1       853       851      184       41
## 2  2013     1     1       290       338     1134      213
```

```
## 3  2013     1     1       260       263      266       46
## 4  2013     1     1       157       174      213       60
## 5  2013     1     1       216       222      708      121
## # i 3,301 more rows
```

```
popular_dests <- flights %>%           # all groups bigger than a threshold
  group_by(dest) %>%
  filter(n() > 10000)
nrow(popular_dests)
```

```
## [1] 131440
```

```
popular_dests %>%                      # standardise to per-group metrics
  filter(arr_delay > 0) %>%
  mutate(prop_delay = arr_delay / sum(arr_delay)) %>%
  select(year:day, dest, arr_delay, prop_delay)
```

```
## # A tibble: 50,245 x 6
## # Groups:   dest [9]
##    year month   day dest  arr_delay prop_delay
##   <int> <int> <int> <chr>     <dbl>      <dbl>
## 1  2013     1     1 MIA          33  0.000235
## 2  2013     1     1 ORD          12  0.0000424
## 3  2013     1     1 FLL          19  0.0000938
## 4  2013     1     1 ORD           8  0.0000283
## 5  2013     1     1 LAX           7  0.0000344
## # i 50,240 more rows
```